

P0398R0: Core issue 1518: Explicit default constructors and copy-list-initialization

Introduction

As directed by CWG in Oulu, a class that declares an explicit default constructor or has inherited constructors should cease being an aggregate.

Wording

Change in 8.5.1 [dcl.init.aggr] paragraph 1:

An aggregate is an array or a class (Clause 9) with

- no user-provided, **explicit, or inherited** constructors (12.1) ~~(including those inherited (7.3.3) from a base class),~~
- ...

Change in 13.3.1.3 [over.match.ctor] paragraph 1:

When objects of class type are direct-initialized (8.5), copy-initialized from an expression of the same or a derived class type (8.5), or default-initialized (8.5), overload resolution selects the constructor. For direct-initialization or default-initialization **that is not in the context of copy-initialization**, the candidate functions are all the constructors of the class of the object being initialized. For copy-initialization, the candidate functions are all the converting constructors (12.3.1) of that class. The argument list is the *expression-list* or *assignment-expression* of the initializer.

Change in 12.3.1 [class.conv.ctor] paragraphs 1 and 2:

A constructor declared without the *function-specifier* *explicit* specifies a conversion from the types of its parameters **(if any)** to the type of its class. Such a constructor is called a *converting constructor*. [Example: ...]

[Note: An explicit constructor constructs objects just like non-explicit constructors, but does so only where the direct-initialization syntax (8.5) or where casts (5.2.9, 5.4) are explicitly used; see also 13.3.1.4. A default constructor may be an explicit constructor; such a constructor will be used to perform default-initialization or value-initialization (8.5). [Example:

```
struct Z {  
    explicit Z();  
    explicit Z(int);  
    explicit Z(int, int);  
};  
Z a; // OK: default-initialization performed  
Z b{}; // OK: direct initialization syntax used  
Z c = {}; // error: copy-list-initialization  
Z a1 = 1; // error: no implicit conversion  
Z a3 = Z(1); // OK: direct initialization syntax used  
Z a2(1); // OK: direct initialization syntax used  
Z* p = new Z(1); // OK: direct initialization syntax used  
Z a4 = (Z)1; // OK: explicit cast used  
Z a5 = static_cast<Z>(1); // OK: explicit cast used  
Z a6 = { 3, 4 }; // error: no implicit conversion
```

-- end example] -- end note]